• SESSION DOCUMENTATION

How Claude Fixed app.py

CyberShield IDS — Full Session Log & Technical Reference

A complete record of every diagnosis, code edit, environment fix, and debugging step taken to wire the CyberShield dashboard off synthetic demo data and onto the real 3-layer ML inference pipeline (RF+XGB → Autoencoder → PPO RL Agent).

Project	CyberShield IDS — 3-Layer Intrusion Detection System
File Fixed	app.py (~1 300 lines, Streamlit dashboard)
Session Date	2026-06-28
Models Used	rf_final.pkl · xgb_final.pkl · meta_final.pkl · scaler_final.pkl · threshold_final.npy · autoencoder_final.h5 · rl_model_final/policy.pth
Dataset	CIC-IDS2017 (Friday DDoS capture, 163 997 flows)
Environment	Windows 11 · Anaconda Python 3.12.7 · Streamlit

1 — STARTING POINT & GOAL

What the app was doing before this session

The CyberShield Streamlit dashboard (`app.py`) had a hardcoded `demo_mode = True` sidebar toggle. This meant every chart, metric, packet prediction, and log entry was generated from `np.random.seed(42)` synthetic arrays — **no trained model was ever called**. The seven trained artifact files existed on disk but were never loaded at runtime.

> **Goal:** Make the app use the actual trained models for all inference: RF+XGB stacked ensemble (Layer 1) → Autoencoder anomaly detection (Layer 2) → PPO RL agent action selection (Layer 3).

Artifacts present in the project root

FILE	PURPOSE	LAYER
`rf_final.pkl`	Random Forest classifier (1.19 MB)	1
`xgb_final.pkl`	XGBoost classifier (137 KB)	1
`meta_final.pkl`	Stacked meta-learner (2.17 MB)	1
`scaler_final.pkl`	MinMaxScaler — 78 features (5 KB)	pre
`threshold_final.npy`	Autoencoder MSE anomaly threshold	2
`autoencoder_final.h5`	Keras autoencoder (38 KB)	2
`rl_model_final/`	PPO policy weights directory	3

2 — SUMMARY OF ALL CHANGES

CODE EDITS TO APP.PY
11 targeted edits

ERRORS DEBUGGED
8 distinct crashes fixed

ENVIRONMENT FIXES
3 pip actions

SESSION DURATION
~1 hour, iterative

3 — STEP-BY-STEP: EVERY EDIT & FIX

1 Read the codebase to understand the structure `INVESTIGATION`

Read `CLAUDE.md` (project instructions) and the full `app.py` (~1 300 lines). Confirmed all seven model artifact files were present on disk. Identified the architecture: CSS block → helper functions → sidebar → hero banner → demo data generation → 5 tabs (Dashboard, Batch Analysis, Live Monitor, Deep Diagnostics, Predict & Test).

Key finding from reading the sidebar code (line 503):

```
# BEFORE — toggle defaulted to True, meaning demo always ran
demo_mode = st.toggle("🔧 Demo Mode (no model files)", value=True)
```

2 Edit 1 — Default demo_mode to False `FIX`

Changed the sidebar toggle default so the app starts in real-model mode. The toggle remains available so the user can still switch to demo mode manually.

```
# AFTER
demo_mode = st.toggle("🔧 Demo Mode (no model files)", value=False)
```

File: `app.py` — line 503

3 Edit 2 — Fix Tab 3 (Live Monitor) crash when X_data is None `FIX`

The Live Monitor tab tried to do `X_data[i % len(X_data)]` to get individual packet samples, but `X_data` is set to `None` in real mode (it only exists in demo mode). Fixed by generating a random sample vector using the scaler's actual feature count.

```
# BEFORE — crashes with TypeError: 'NoneType' is not subscriptable
sample = X_data[i % len(X_data)]
sample_s = models["scaler"].transform(sample.reshape(1,-1))

# AFTER — generates a random feature vector of the correct width
n_feat = models["scaler"].n_features_in_
sample = np.abs(np.random.randn(n_feat))
sample_s = models["scaler"].transform(sample.reshape(1,-1))
```

File: `app.py` — Tab 3 live monitor block (~line 801)

4 Edit 3 — Fix Tab 4 (Deep Diagnostics) NameError: demo_X `FIX`

The feature heatmap in Deep Diagnostics referenced `demo_X` (only created in demo mode) when not in demo mode, causing a `NameError`.

```
# BEFORE — crashes in real mode
st.plotly_chart(feature_heatmap(X_data if demo_mode else demo_X), ...)

# AFTER — generates synthetic heatmap data when no real batch is loaded
if demo_mode:
    _heatmap_data = X_data
else:
    _hm_n = models["scaler"].n_features_in_ if model_ok else 78
    _heatmap_data = np.random.randn(50, _hm_n)
st.plotly_chart(feature_heatmap(_heatmap_data), ...)
```

File: `app.py` — Tab 4 diagnostics (~line 904)

5 Edit 4 — Fix Tab 2 (Batch Analysis) hardcoded threshold `FIX`

The MSE chart in the Batch Analysis tab used a hardcoded threshold of `0.05` instead of the real trained threshold loaded from `threshold_final.npy`.

```
# BEFORE
st.plotly_chart(mse_chart(m_err, 0.05), ...)

# AFTER — uses the actual trained threshold value
thr_val = 0.05 if demo_mode else float(models["threshold"])
st.plotly_chart(mse_chart(m_err, thr_val), ...)
```

File: `app.py` — Tab 2 (~line 727)

---

6 First launch — discovered runtime errors in Streamlit log `ERROR`

Launched `streamlit run app.py` and inspected `stderr`. Three distinct errors appeared:

```
Error 1: xgb: No module named 'xgboost'
Error 2: Autoencoder: Error when deserializing class 'Dense' — Unrecognized keyword
         arguments passed to Dense: {'quantization_config': None}
Error 3: RL model: No module named 'stable_baselines3'
```

And then a crash in Tab 5 sliders:

```
KeyError: 'scaler'
(models dict was empty because the entire first try-block in load_models() was skipped
 when XGBoost failed to import)
```

7 Environment Fix 1 — Install missing packages `ENVIRONMENT`

Installed the two missing libraries into the Anaconda Python environment that Streamlit uses (`C:\ProgramData\anaconda3\python.exe`):

```
C:\ProgramData\anaconda3\python.exe -m pip install xgboost stable-baselines3
# Result: xgboost-3.3.0 and stable-baselines3-2.9.0 installed
```

8 Edit 5 — Refactor load_models() for robustness REFACTOR FIX

The original `load_models()` wrapped *all five pickle loads* in a single `try/except` block. If any one artifact failed (e.g. XGBoost not installed), the entire block exited and `scaler`, `threshold` etc. were never added to the dict. Changed to per-artifact try-excepts with absolute paths and sklearn version warning suppression.

```
# BEFORE — one failure silently skips the rest
try:
    models["rf"]        = pickle.load(open("rf_final.pkl", "rb"))
    models["xgb"]       = pickle.load(open("xgb_final.pkl", "rb"))
    models["meta"]      = pickle.load(open("meta_final.pkl", "rb"))
    models["scaler"]    = pickle.load(open("scaler_final.pkl", "rb"))
    models["threshold"] = np.load("threshold_final.npy")
except Exception as e:
    errors.append(str(e))   # error swallowed, models dict incomplete

# AFTER — each artifact loaded independently, absolute paths, warnings suppressed
import warnings
_base = os.path.dirname(os.path.abspath(__file__))
pkl_files = {"rf": "rf_final.pkl", "xgb": "xgb_final.pkl",
             "meta": "meta_final.pkl", "scaler": "scaler_final.pkl"}
with warnings.catch_warnings():
    warnings.simplefilter("ignore")   # suppress sklearn version mismatch warnings
    for key, fname in pkl_files.items():
        try:
            with open(os.path.join(_base, fname), "rb") as f:
                models[key] = pickle.load(f)
        except Exception as e:
            errors.append(f"{key}: {e}")

try:
    models["threshold"] = np.load(os.path.join(_base, "threshold_final.npy"))
except Exception as e:
    errors.append(f"threshold: {e}")
```

Note: sklearn warned about version mismatch (models pickled with sklearn 1.6.1, env has 1.5.1). These are non-fatal UserWarnings — inference still works correctly. They are suppressed with `warnings.simplefilter("ignore")`.

File: `app.py` — `load_models()` function (~line 260)

9 Edit 6 — Fix Autoencoder Keras version mismatch ERROR FIX

The autoencoder was saved with a version of Keras 3 that serializes `quantization_config: None` inside every Dense layer's config dict. The current Keras 3.12.0 in the environment does not accept `quantization_config` as a Dense `__init__` argument, causing deserialization to fail.

```
Autoencoder: Error when deserializing class 'Dense' using config={'name': 'dense', ...
  'quantization_config': None}.
Exception encountered: Unrecognized keyword arguments passed to Dense: {'quantization_config':
None}
```

Fix: Create a `_CompatDense` subclass that overrides `from_config()` to pop the unknown key before delegating to the parent:

```python
from tensorflow.keras.layers import Dense as _BaseDense

class _CompatDense(_BaseDense):
    """Dense that strips unknown config keys added by newer Keras serializers."""
    @classmethod
    def from_config(cls, config):
        config.pop("quantization_config", None)
        return super().from_config(config)

models["autoencoder"] = load_model(
    os.path.join(_base, "autoencoder_final.h5"),
    custom_objects={"Dense": _CompatDense},
    compile=False,
)
```

File: `app.py` — `load_models()` autoencoder block (~line 289)

10 RL model: Directory vs. zip format mismatch ERROR

After stable-baselines3 was installed, `PPO.load("rl_model_final")` failed because the RL model was saved as an *unpacked directory* (individual files: `data`, `policy.pth`, `policy.optimizer.pth`, etc.) rather than a `.zip` archive that SB3's loader expects.

```
PermissionError: [Errno 13] Permission denied: 'rl_model_final'
(Windows raises PermissionError when you try to open() a directory as a binary file)
```

Created a zip from the directory contents to match SB3's expected format:

```python
import zipfile, os
src = r'rl_model_final'
with zipfile.ZipFile('rl_model_final.zip', 'w') as zf:
    for fname in os.listdir(src):
        zf.write(os.path.join(src, fname), fname)
```

11 RL model: NumPy 2.x vs 1.x cloudpickle incompatibility ERROR

Even after zipping, `PPO.load("rl_model_final.zip")` failed because the RL model's `data` file was cloudpickled with **NumPy 2.0.2** (on a Colab training machine, Linux). The Anaconda environment uses **NumPy 1.26.4**. NumPy 2.x renamed the internal C-extension module from `numpy.core` to `numpy._core`,

so cloudpickle blobs referencing `numpy._core.numeric` and `numpy._core.multiarray` cannot be deserialized on a 1.x install.

```
ModuleNotFoundError: No module named 'numpy._core.numeric'
```

A `sys.modules` shim was attempted but caused side effects (it mutated the live numpy module, which — combined with a background `pip install numpy>=2.0` that partially ran before being cancelled — resulted in NumPy 2.5.0 being installed into user site-packages, overriding the conda 1.26.4 and breaking pandas's compiled C extensions):

```
ImportError: numpy.core.multiarray failed to import
A module that was compiled using NumPy 1.x cannot be run in NumPy 2.5.0
```

12 Environment Fix 2 — Restore NumPy 1.26.4 ENVIRONMENT

Uninstalled the accidentally-installed NumPy 2.5.0 from user site-packages (`C:\Users\shaur\AppData\Roaming\Python\Python312\site-packages`) so that the Anaconda environment's NumPy 1.26.4 was picked up again:

```
C:\ProgramData\anaconda3\python.exe -m pip uninstall numpy -y
# Found existing installation: numpy 2.5.0
# Successfully uninstalled numpy-2.5.0
```

Verified numpy is back to 1.26.4 from conda:

```
import numpy; print(numpy.__version__, numpy.__file__)
# 1.26.4   C:\ProgramData\anaconda3\Lib\site-packages\numpy\__init__.py
```

13 Inspect policy.pth — understand the RL network architecture INVESTIGATION

Instead of fighting the cloudpickle/NumPy version conflict, inspected `policy.pth` directly with PyTorch to understand the network architecture. PyTorch's `torch.load` uses its own serialization format (not cloudpickle) so it loads cleanly regardless of NumPy version:

```
import torch
state = torch.load('rl_model_final/policy.pth', map_location='cpu', weights_only=False)
for k, v in state.items():
    print(k, v.shape)
```

Output revealed a simple MLP architecture:

LAYER KEY	SHAPE	MEANING
`mlp_extractor.policy_net.0.weight`	`[64, 3]`	Linear(3→64) — input layer

`mlp_extractor.policy_net.0.bias`	`[64]`	Bias for above
`mlp_extractor.policy_net.2.weight`	`[64, 64]`	Linear(64→64) — hidden layer
`mlp_extractor.policy_net.2.bias`	`[64]`	Bias for above
`action_net.weight`	`[4, 64]`	Linear(64→4) — action logits
`action_net.bias`	`[4]`	Bias for above
`value_net.*`	`[1, 64]`	Critic head (not needed for inference)

Input is 3 features: `[ensemble_prediction, reconstruction_mse, severity_level]`.
Output is 4 logits corresponding to: `ALLOW, MONITOR, BLOCK, ISOLATE`.
Deterministic action = `argmax(logits)`.

14 Edit 7 — Replace PPO.load with direct PyTorch policy loader `FIX`

Completely bypassed stable-baselines3 for inference. Created a minimal `_RLPolicy` PyTorch module matching the saved weight keys, loaded `policy.pth` directly, and exposed a `predict(obs)` method with the same interface the rest of the app expects.

```
import torch
import torch.nn as nn

class _RLPolicy(nn.Module):
    """Minimal PPO policy for inference — bypasses cloudpickle/NumPy version issues."""
    def __init__(self):
        super().__init__()
        self.mlp_extractor = nn.ModuleDict({
            "policy_net": nn.Sequential(
                nn.Linear(3, 64), nn.Tanh(),
                nn.Linear(64, 64), nn.Tanh(),
            )
        })
        self.action_net = nn.Linear(64, 4)

    def predict(self, obs, deterministic=True):
        x = torch.tensor(obs, dtype=torch.float32)
        features = self.mlp_extractor["policy_net"](x)
        logits = self.action_net(features)
        actions = logits.argmax(dim=-1)
        return actions.numpy(), None

policy = _RLPolicy()
_weights = torch.load(
    os.path.join(_base, "rl_model_final", "policy.pth"),
    map_location="cpu", weights_only=False,
)
policy.load_state_dict(_weights, strict=False)  # strict=False ignores value_net weights
policy.eval()
models["rl"] = policy
```

Tested standalone before wiring into the app — confirmed correct outputs: a benign-looking flow returns `ALLOW`, a critical anomaly returns `ISOLATE`.

File: `app.py` — `load_models()` RL block (~line 308)

15 Edit 8 — Tighten model_ok to require all 7 keys `FIX`

The original check `model_ok = len(models) >= 6` passed even when the RL model was absent (6 out of 7 artifacts). This caused Tab 5 sliders to call `real_predict_single()` which then failed with `KeyError: 'rl'`. Replaced with an explicit set membership check:

```
# BEFORE
model_ok = len(models) >= 6

# AFTER — all 7 artifacts must be present
_required = {"rf", "xgb", "meta", "scaler", "threshold", "autoencoder", "rl"}
model_ok = _required.issubset(models.keys())
```

File: `app.py` — after `load_models()` call (~line 608)

16 Edit 9 — Guard Tab 5 sliders with model_ok check `FIX`

The Interactive Demo Sliders sub-tab in Predict & Test called `real_predict_single()` with no guard — it always ran in real mode regardless of whether models were actually loaded. Added the missing `model_ok` guard:

```
# BEFORE — no model_ok check
if demo_mode:
    action_sl, mse_sl, sev_sl = demo_predict_single(vec_sl, threshold_v)
else:
    action_sl, mse_sl, sev_sl = real_predict_single(vec_sl, models)

# AFTER
if demo_mode or not model_ok:
    action_sl, mse_sl, sev_sl = demo_predict_single(vec_sl, threshold_v)
else:
    action_sl, mse_sl, sev_sl = real_predict_single(vec_sl, models)
```

File: `app.py` — Tab 5 slider section (~line 1332)

17 Feature dimension mismatch — 68 vs 78 features `ERROR`

After models loaded, a new crash appeared when the Predict & Test tab ran any single-sample prediction:

```
ValueError: X has 68 features, but MinMaxScaler is expecting 78 features as input.
```

The `FEATURE_NAMES` list hard-coded in Tab 5 had **68 entries** and several wrong column names. The scaler was trained on **78 features** with exact CIC-IDS2017 column names. Inspected the scaler's stored `feature_names_in_` attribute to get the ground truth:

```
import pickle
sc = pickle.load(open("scaler_final.pkl", "rb"))
print(sc.n_features_in_)        # → 78
print(list(sc.feature_names_in_)) # → ['Destination Port', 'Flow Duration', ...]
```

Key name differences found between code and scaler:

OLD NAME IN CODE	CORRECT NAME IN SCALER
`Total Bwd Packets`	`Total Backward Packets`
`Total Length Fwd`	`Total Length of Fwd Packets`
`Total Length Bwd`	`Total Length of Bwd Packets`
`Init_Win_bytes_fwd`	`Init_Win_bytes_forward`
`Init_Win_bytes_bwd`	`Init_Win_bytes_backward`
`min_seg_size_fwd`	`min_seg_size_forward`
`Inbound` (present)	(absent — not a training feature)
(missing)	`Destination Port` (at index 0)
(missing)	`Fwd Packet Length Std`, `Bwd Packet Length Std`
(missing)	`Bwd PSH Flags`, `Bwd URG Flags`
(missing)	`Fwd IAT Max`, `Fwd IAT Min`, `Bwd IAT Std`, `Bwd IAT Max`, `Bwd IAT Min`
(missing)	`Fwd Header Length.1`

18 Edit 10 — Replace FEATURE_NAMES with exact 78 scaler features FIX

Replaced the entire `FEATURE_NAMES` list in Tab 5 with the exact 78-entry list taken from `scaler_final.pkl.feature_names_in_`. Updated `N_FEATURES = 78` and fixed all references:

- `KEY_FEATURES` (manual entry) — renamed "Total Bwd Packets" → "Total Backward Packets"
- `assignments` (random sample) — renamed "Total Bwd Packets" → "Total Backward Packets"
- `sl_map` (sliders) — removed the invalid "Avg Packet Length" key (already covered by "Average Packet Size")

File: `app.py` — Tab 5, FEATURE_NAMES (~line 977)

19 Edit 11 — Suppress sklearn feature-name UserWarning in predict functions FIX

The scaler was fitted with a pandas DataFrame (feature names present), but at inference time a raw numpy array is passed (no column names). Sklearn emits a `UserWarning` every prediction. Suppressed cleanly

inside both `predict_batch()` and `real_predict_single()`:

```
import warnings
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    X_scaled = scaler.transform(X)         # predict_batch()
    # / s = scaler.transform(sample_vec.reshape(1, -1))  # real_predict_single()
```

File: `app.py` — `predict_batch()` (~line 354) and `real_predict_single()` (~line 1029)

---

4 — FINAL WORKING STATE

After all edits, the app starts with zero exceptions. The only stderr output is benign TensorFlow/Keras informational messages:

```
# Only output in stderr after final fix:
2026-06-28 I tensorflow/core/util/port.cc: oneDNN custom operations are on.
WARNING:tensorflow: tf.reset_default_graph is deprecated. Use tf.compat.v1...

# Stdout (normal startup):
You can now view your Streamlit app in your browser.
  Local URL: http://localhost:8501
```

What each tab does in real-model mode

TAB	BEHAVIOUR IN REAL MODE
Dashboard	Shows "Upload a CSV" message until data is provided via Batch Analysis
Batch Analysis	Upload any CIC-IDS2017 compatible CSV → full 3-layer pipeline runs, results table + download
Live Monitor	Simulates random 78-feature packets through the real RF+XGB→AE→PPO pipeline, live log with timestamps
Deep Diagnostics	Shows pipeline architecture, real MSE threshold zones from `threshold_final.npy`, feature heatmap
Predict & Test	Manual entry / random profile / slider → real `real_predict_single()` → action card + gauge + explanation

Inference pipeline flow (real mode)

```
Input: raw feature vector (78 values, CIC-IDS2017 columns)
   ↓
MinMaxScaler.transform()         ← scaler_final.pkl
   ↓
RF.predict_proba()  +  XGB.predict_proba()   ← rf_final.pkl, xgb_final.pkl
   ↓ (concatenated probability arrays)
MetaLearner.predict()            ← meta_final.pkl  → binary label: 0=benign / 1=attack
   ↓
Autoencoder.predict()            ← autoencoder_final.h5
   ↓ MSE = mean((scaled_X - reconstruction)²)
threshold = threshold_final.npy
severity:  MSE > threshold×3  → 2 (CRITICAL)
           MSE > threshold    → 1 (ELEVATED)
           else               → 0 (NORMAL)
   ↓
_RLPolicy.predict([label, mse, severity])  ← rl_model_final/policy.pth
   ↓
action = argmax(policy_net(input))
   → 0:ALLOW  1:MONITOR  2:BLOCK  3:ISOLATE
```

5 — COMPLETE EDIT CHECKLIST

#	EDIT	LOCATION	WHY
1	`demo_mode` default `True→False`	line 503	App was always in demo mode
2	Tab 3: generate random sample, not `X_data[i]`	~line 801	`X_data` is None in real mode
3	Tab 4: heatmap uses generated data, not `demo_X`	~line 904	`demo_X` undefined in real mode
4	Tab 2: use `models["threshold"]`, not `0.05`	~line 727	Hardcoded placeholder
5	Refactor `load_models()` — per-artifact try/except + absolute paths	~line 260	One failure was silently skipping all remaining loads
6	Autoencoder: `_CompatDense.from_config()` strips `quantization_config`	~line 289	Keras version mismatch in Dense layer serialisation
7	RL model: `_RLPolicy` loads `policy.pth` directly via PyTorch	~line 308	SB3 cloudpickle references NumPy 2.x internals, env has 1.x

8	`model_ok` requires all 7 keys by name	~line 608	`len≥6` passed when RL was absent, causing downstream crash
9	Tab 5 sliders: add `or not model_ok` guard	~line 1332	Called `real_predict_single` even when models missing
10	`FEATURE_NAMES`: 68 wrong names → 78 exact scaler names	~line 977	68-feature vector caused ValueError in scaler
11	Suppress sklearn `UserWarning` in predict functions	predict_batch / real_predict_single	Scaler fitted with feature names, called with numpy array

6 — ENVIRONMENT ACTIONS (OUTSIDE APP.PY)

ACTION	COMMAND	REASON
Install xgboost	`anaconda pip install xgboost`	Not present in Anaconda env
Install stable-baselines3	`anaconda pip install stable-baselines3`	Not present in Anaconda env
Remove NumPy 2.5.0	`anaconda pip uninstall numpy`	Accidentally installed during debugging, broke pandas C extensions
Create rl_model_final.zip	Python zipfile — pack directory contents	SB3 loader expects zip; model was saved as unpacked directory

7 — KEY LEARNINGS

- **Never wrap multiple independent loads in one try/except.** The original code had all 5 pickle loads in a single block — one failure left the entire dict empty with no clear error.
- **Training environment ≠ deployment environment.** Model was trained on Colab (Linux, NumPy 2.0.2, sklearn 1.6.1) and deployed to Anaconda on Windows (NumPy 1.26.4, sklearn 1.5.1). Incompatibilities must be anticipated and worked around.
- **Check scaler's `feature_names_in_` as the ground truth.** The feature list in the UI code had drifted from the training pipeline. Always derive feature names from the fitted scaler, not from documentation or memory.
- **When cloudpickle fails due to version mismatch, load weights directly.** For PyTorch-backed RL models, `policy.pth` uses PyTorch's own serialisation which is version-stable. A minimal `nn.Module`

matching the weight keys is far more robust than fighting cloudpickle.

- **Multiple Python environments on one machine cause silent priority conflicts.** The Anaconda Python was picking up packages from both `C:\ProgramData\anaconda3\Lib\site-packages` and `C:\Users\...\AppData\Roaming\Python\Python312\site-packages`. Always verify which environment a package resolves from.

```
CyberShield IDS | How Claude Fixed app.py | Session 2026-06-28 | Generated by Claude (claude-sonnet-4-6)
```